Object Oriented Design Patterns II (JIAT/OODP II / EX / 01)

වීඩියෝව **Record** කිරීම සඳහා මූලික උපදෙස් (**General Guidelines & Setup**)

- **කාල සීමාව (Duration):** විනාඩි 10 ත් 15 ත් අතර (Strictly 10–15 Minutes).
- **වීඩියෝ Format එක:** MP4 (Resolution: 1080p Full HD හෝ 720p).
- **Full-Stack Architecture එක පෙන්වීම:**
  1. **MySQL Database (`smartcity360_db`):** MySQL Workbench / CLI එකේ tables 5 පෙන්වන්න.
  2. **Spring Boot REST Backend:** Terminal එකේ `mvn spring-boot:run` කර port 8080 එකේ run වන බව පෙන්වන්න.
  3. **Interactive Modern Dashboard UI:** Browser එකෙන් `http://localhost:8080` හෝ JavaFX window එකෙන් UI views පෙන්වන්න.
- **භාෂාව (Language):** Presentation එක සම්පූර්ණයෙන්ම **English** වලින් සිදු කළ යුතුය (කොළ පැහැති **Speech Box** හි ඇති script එක කියවන්න).
- **තිරයේ කළ යුතු ක්‍රියාකාරකම්:** කහ පැහැති **Activity Box** හි Sinhala & English වලින් පියවරෙන් පියවර දක්වා ඇත.

Scene 1: Introduction, 3-Tier Architecture & MySQL Database 00:00 - 02:00 (2.0 Mins)

තිරයේ කළ යුතු දේ (**On-Screen Activity**):
• MySQL Workbench හෝ Terminal එකෙන් MySQL database එක (`smartcity360_db`) සහ එහි ඇති tables 5 (`citizen_complaints`, `energy_telemetry_logs`, ආදිය) පෙන්වන්න.
• Terminal එකේ `mvn spring-boot:run` කර Backend එක `http://localhost:8080` හි start වන අයුරු පෙන්වන්න.
• Browser / JavaFX window එකෙන් **SmartCity360 Modern Dashboard** එක විවෘත කර "All systems operational" pill එක සහ Cards 5 පෙන්වන්න.

මයික්‍රෆෝනයෙන් කිව යුතු දෙබස් (**Spoken Narration Script**): "Good day, Respected Evaluators. My name is Sadeepa Sri Rohanasinghe, and today I am demonstrating the SmartCity360 Municipal Management Platform, developed for the Object Oriented Design Patterns II Final Assessment.

As a Senior Software Architect at UrbanTech Solutions, I have architected an enterprise 3-tier full-stack system. At the data layer, we have MySQL 8.0 running the `smartcity360_db` database with tables created by Hibernate. In the middle tier, we have a robust Spring Boot 3.x REST API hosting our five Gang of Four design patterns: Decorator, Composite, Chain of Responsibility, Flyweight, and Interpreter. At the presentation tier, we have a modern, responsive UI dashboard.

Let us now walk through each design pattern in action across our full-stack architecture."

Pro-Tip: Spring Boot console එකේ "Started SmartCity360Application" log එක සහ MySQL database tables පැහැදිලිව පෙන්වන්න.

Scene 2: Part A – Dynamic Security Wrappers (Decorator Pattern) 02:00 - 04:30 (2.5 Mins)

තිරයේ කළ යුතු දේ (On-Screen Activity):
• Dashboard එකෙන් **Part A: Dynamic Security Wrappers** card එක click කරන්න.
• "Energy Meter Raw Data Stream" text area එක පෙන්වා, Encryption, Compression, සහ Authentication toggle switches 3 ම ON කරන්න.
• **Process Stream** → button එක click කරන්න.
• Spring Boot REST API එක මඟින් Decorator pipeline එක හරහා payload එක wrap වී, verify වී, MySQL `energy_telemetry_logs` table එකට save වන output එක පෙන්වන්න.

මයික්‍රෆෝනයෙන් කිව යුතු දෙබස් (Spoken Narration Script): "In Part A, we demonstrate dynamic security wrapping for energy telemetry streams using the Decorator Pattern.

Energy meters transmit high-frequency electrical usage data that must be secured with AES-128 Encryption, GZIP Compression, and HMAC Authentication. If we used static inheritance, supporting all optional permutations would produce an explosion of eight subclasses.

By applying the Decorator Pattern, our base component 'MeterDataStream' remains completely unaltered, upholding the Open/Closed Principle. In our UI, we have toggled on Encryption, Compression, and Authentication.

When we click 'Process Stream', our Spring Boot backend dynamically assembles the decorator pipeline. Outbound, the payload is signed with a SHA-256 HMAC token, compressed, and encrypted with AES-128. Inbound, the stream unwraps in reverse: decrypted, decompressed, and authenticated against tampering. As displayed on screen, data integrity is 100% verified, and the telemetry audit log is persisted in MySQL."

Pro-Tip: Toggles on/off කර runtime එකේදී dynamic composition සිදුවන බව පෙන්වන්න.

Scene 3: Part B – Smart Zone Hierarchy (Composite Pattern) 04:30 - 07:00 (2.5 Mins)

තිරයේ කළ යුතු දේ (**On-Screen Activity):**
• Navigation එකෙන් **Part B: Smart Zone Hierarchy** view එකට යන්න.
• Breadcrumb trail එක පෙන්වන්න: `Metro Colombo → Districts → Streets → Buildings`.
• **Execute Recursive Power Shutdown** button එක click කරන්න → Colombo 01 power draw 0 වී, city power draw 2200 kWh සිට 340 kWh දක්වා අඩුවන අයුරු පෙන්වන්න.
• **Execute Recursive Usage Report** click කර power නැවත restore කර පෙන්වන්න.

මයික්‍රෆෝනයෙන් කිව යුතු දෙබස් (**Spoken Narration Script):** "Moving to Part B: Smart Zone Hierarchy, modeled using the Composite Pattern.

City infrastructure is organized in a natural spatial tree: City contains Districts, Districts contain Streets, and Streets contain individual Buildings. We need commands like 'shutdownPower()' and 'reportUsage()' to execute uniformly whether invoked on a single building or an entire district.

In our design, 'Building' is a Leaf node, while 'ZoneComposite' holds a collection of children and recursively cascades operations.

Observing the screen: The total city power consumption starts at 2,200 kilowatt-hours. When we click 'Execute Recursive Power Shutdown' on Colombo 01, the command automatically cascades through York Street and constituent buildings. Colombo 01 drops to 0 kWh, reducing metro grid load to 340 kWh, while Colombo 05 remains operational. Clicking 'Execute Recursive Usage Report' recalculates the bottom-up consumption and restores full grid power. This demonstrates seamless polymorphic recursion without client-side type checking."

Pro-Tip: Single building සහ whole district දෙකම එකම interface එකකින් uniform ලෙස ක්‍රියා කරන බව අවධාරණය කරන්න.

Scene 4: Part C – Citizen Complaint Escalation (Chain of Responsibility) 07:00 - 09:30 (2.5 Mins)

තිරයේ කළ යුතු දේ (**On-Screen Activity):**
• Dashboard එකෙන් **Part C: Complaint Escalation** view එකට යන්න.
• Complaint description එකක් enter කරන්න (උදා: "Transformer explosion and power outage").
• Severity Level එක **Critical** ලෙස select කරන්න.
• **Submit & Escalate** → button එක click කරන්න.
• Local Staff → Supervisor → City General Manager වෙත escalate වී complaint එක resolve වන Audit Log එක පෙන්වන්න.
• MySQL table `citizen_complaints` හි අදාළ record එක save වී ඇති අයුරු පෙන්වන්න.

මයික්‍රෆෝනයෙන් කිව යුතු දෙබස් (**Spoken Narration Script):** "Now let us examine Part C: Citizen Complaint Escalation, implemented with the Chain of Responsibility Pattern.

Citizen complaints range from minor potholes to municipal emergencies. Directly hardcoding routing logic into the submission portal creates tight coupling. The Chain of Responsibility decouples the sender from the receiver by passing requests along an escalation chain of candidate handlers.

In our UI: We enter a critical complaint regarding a transformer explosion. When we click 'Submit & Escalate', the request hits our Spring Boot endpoint. In the backend, Local Staff checks their authority—it exceeds their tier, so it delegates to the Supervisor, who in turn escalates to the City General Manager. The City General Manager triggers the municipal emergency protocol.

As shown in the audit box and in our MySQL `citizen_complaints` table, the entire escalation path, the resolving authority, and timestamped resolution notes are recorded."

Pro-Tip: Request එකක් drop නොවී MySQL database එකේ audit trail එකක් save වන බව පෙන්වන්න.

Scene 5: Part D – Smart Streetlights Management (Flyweight Pattern)

09:30 - 12:00 (2.5 Mins)

තිරයේ කළ යුතු දේ (**On-Screen Activity):**
• Navigation එකෙන් **Part D: Smart Street Lights** view එකට යන්න.
• Intrinsic State (shared hardware specs & 2KB calibration data) සහ Extrinsic State (GPS coordinates & dimming) විස්තර කරන්න.
• **Run 50,000 Lights Empirical Benchmark** button එක click කරන්න.
• Terminal / UI එකේ Benchmark metrics පෙන්වන්න: Without Flyweight: 108 MB vs With Flyweight: 4.74 MB, **95.6% memory savings**!

මයික්‍රෆෝනයෙන් කිව යුතු දෙබස් (**Spoken Narration Script):** "In Part D, we tackle massive IoT device scaling using the Flyweight Pattern for smart streetlights.

A metropolitan grid operates tens of thousands of streetlights. Every single light fixture shares identical physical specifications: bulb model, wattage, color temperature, and calibration curves. Only their GPS coordinates, power status, and dimming levels are unique.

We separated the state into Intrinsic and Extrinsic. The Intrinsic State—including heavy 2KB spectral calibration arrays—is encapsulated in the immutable 'StreetLightType' flyweight and cached by 'StreetLightFactory'. The Extrinsic State is passed at runtime.

When we click 'Run 50,000 Lights Empirical Benchmark', our backend executes an empirical JVM heap benchmark: Without the Flyweight pattern, duplicating specifications across 50,000 objects

consumes 108.01 megabytes of heap. With the Flyweight pattern, all 50,000 objects share a single cached flyweight instance, slashing memory consumption to just 4.74 megabytes—achieving a verified 95.6% memory reduction."

Pro-Tip: 95.6% memory savings එක සහ JVM Garbage Collection pauses අවම වන බව පවසන්න.

Scene 6: Part E – City Policy Rules DSL (Interpreter Pattern) 12:00 - 13:45 (1.75 Mins)

තිරයේ කළ යුතු දේ (**On-Screen Activity**):
• Dashboard එකෙන් **Part E: City Policy Rules DSL** view එකට යන්න.
• Select Policy Rule dropdown එකෙන් "Night Noise Limit (noise_level > 70 AND hour >= 22)" තෝරන්න.
• Input telemetry query එක පෙන්වන්න (`noise_level: 78, hour: 23`).
• **Evaluate Policy** → button එක click කරන්න → AST Tree එක compile වී, VIOLATION DETECTED වී, "DISPATCH POLICE WARNING" action එක trigger වන අයුරු පෙන්වන්න.

මයික්‍රෆෝනයෙන් කිව යුතු දෙබස් (**Spoken Narration Script**): "For Part E, we address dynamic municipal regulations using the Interpreter Pattern.

City ordinances—such as night noise limits and commercial parking curfews—are modified frequently. Hardcoding these rules into Java if-else statements requires rebuilding and redeploying production servers for every policy amendment.

We designed a Domain-Specific Language (DSL) mini-language. Our 'PolicyParser' translates plain-text rules into an Abstract Syntax Tree of Expression objects. In our UI, we select the Night Noise Rule: 'noise_level > 70 AND hour >= 22'. When evaluated against late-night residential telemetry at 11 PM with 78 decibels, the AST evaluates to TRUE, automatically triggering a police dispatch citation. City managers can introduce new rules dynamically without altering any backend code."

Pro-Tip: Code compile නොකර DSL rule එකක් runtime එකේදී dynamic evaluate වන බව පෙන්වන්න.

Scene 7: Automated Tests, Maven Build Verification & Conclusion 13:45 - 15:00 (1.25 Mins)

තිරයේ කළ යුතු දේ (**On-Screen Activity**):
• Terminal එකේ `mvn test` run කරන්න.
• Automated Tests 6 ම pass වී **BUILD SUCCESS** ලැබෙන අයුරු පෙන්වන්න.
• අවසානයේ කැමරාව/මයික්‍රෆෝනය දෙසට හැරී ඇගයීම් මණ්ඩලයට ස්තූතිවන්ත වන්න.

මයික්‍රෆෝනයෙන් කිව යුතු දෙබස් (**Spoken Narration Script**): "To prove code reliability and verify zero regression, we execute our automated JUnit 5 test suite with 'mvn test'.

As you can observe: All six comprehensive test cases pass with zero failures and zero errors in milliseconds, concluding with Maven BUILD SUCCESS. This validates our Decorator, Composite, Chain of Responsibility, Flyweight, and Interpreter implementations.

In conclusion, SmartCity360 provides an enterprise 3-tier solution featuring a Spring Boot REST API, MySQL database, and modern interactive dashboard UI that demonstrates the practical power of Gang of Four design patterns in solving real-world software architecture challenges.

Thank you very much for your time and evaluation."

Pro-Tip: "Thank you very much for your time and evaluation" යැයි පවසා screen recording එක stop කරන්න.